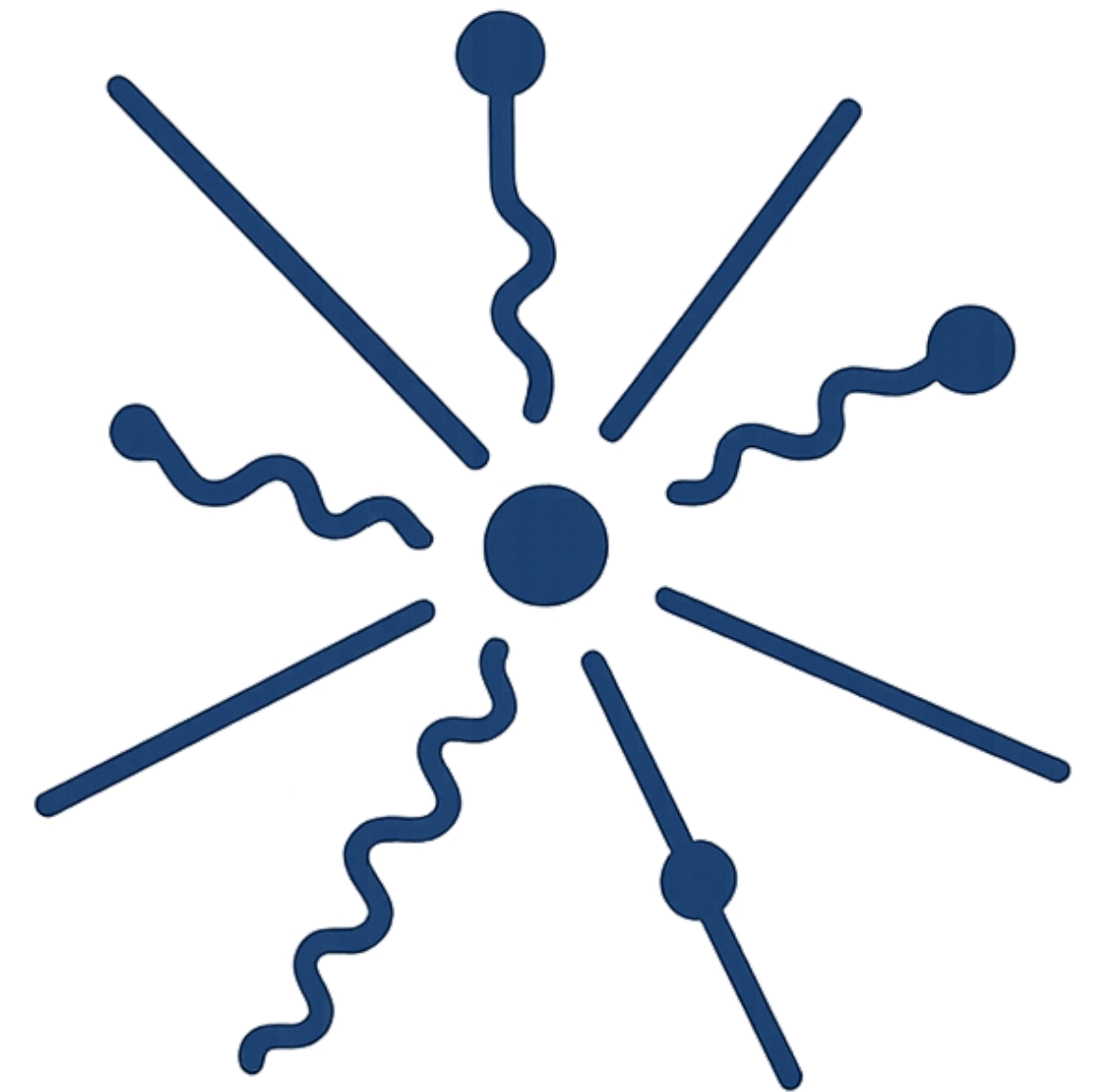
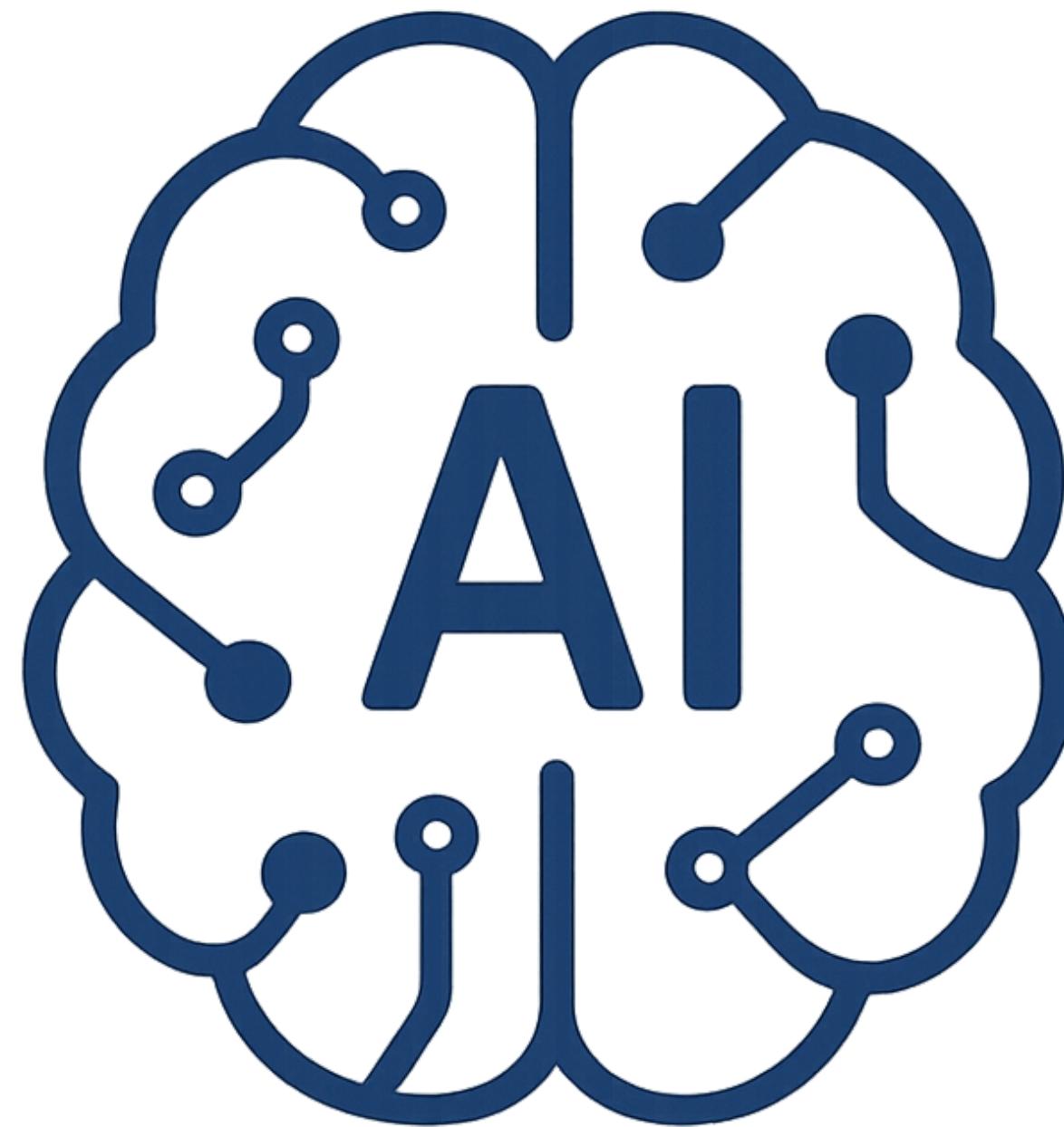
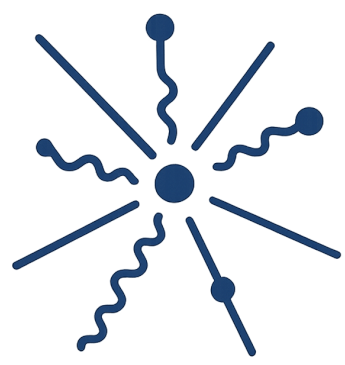
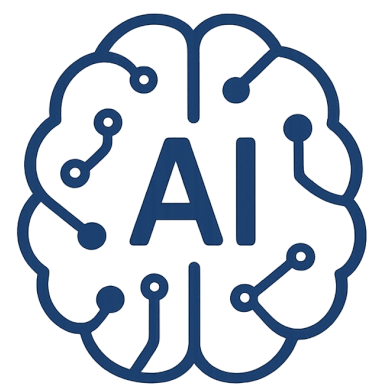


Introduction to

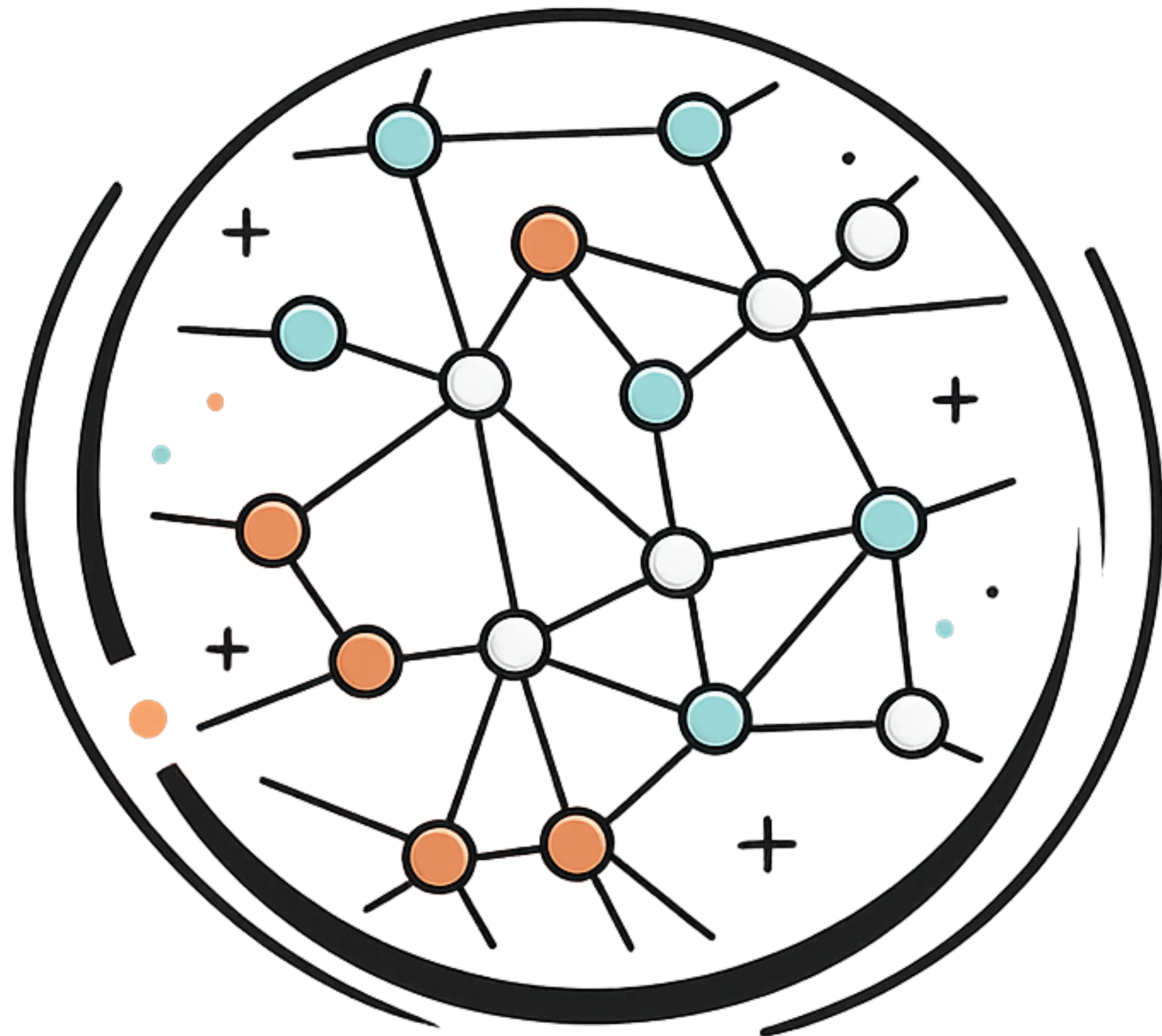
AI-Driven HEP

S. A. Fard - School of Physics (IPM)





AI-Driven HEP 25



Session 25

Graph Neural Network

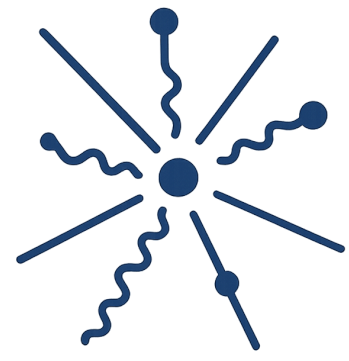
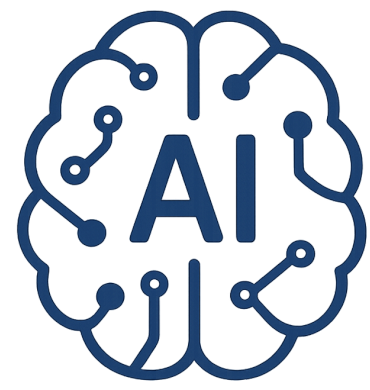


Reference

- [1] Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Springer Nature, 2023.
- [2] Simon J.D. Prince, Understanding Deep Learning, The MIT Press (2025), <https://udlbook.github.io/udlbook/>

<https://physics.ipm.ac.ir/~ansarifard/>





3 Main Network

CNN

(B, N, H, W, C)

Transformer

(B, N, D)

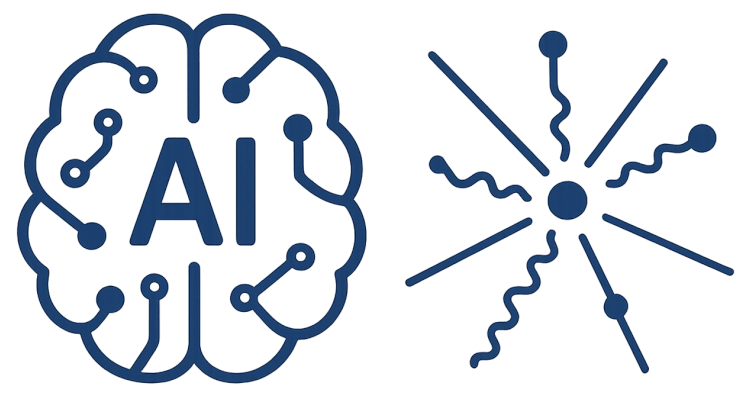
GNN

(B, N, D)

Sequential

(N, D)

Graph

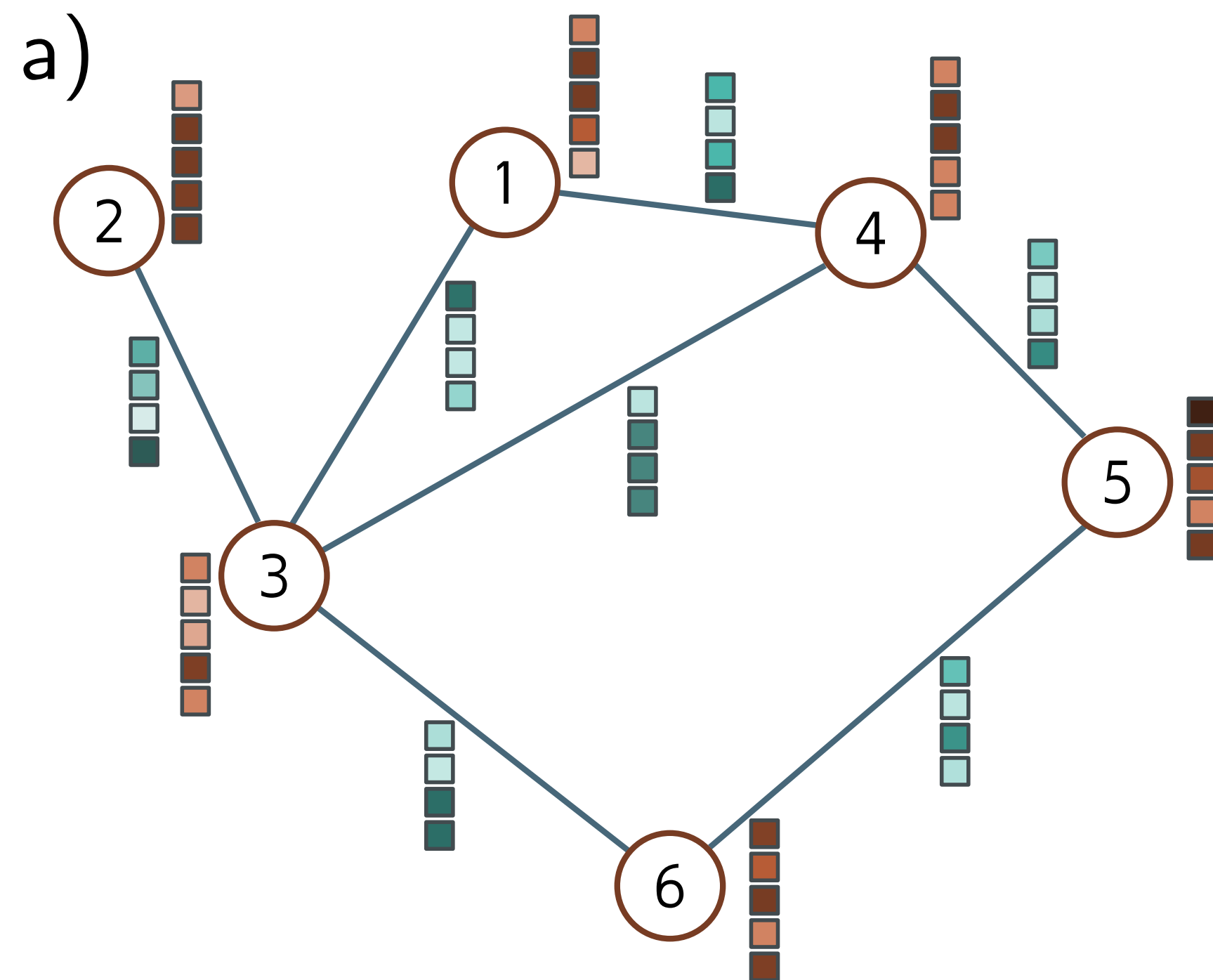


Simple Graph

N node : represented by a vector of D dimensions

Adjacency matrix : shows edges of nodes (similar to positional embedding)

Edge matrix : For simple graph can be ignored



b)

Adjacency matrix, $\mathbf{A}$
 $N \times N$

	1	2	3	4	5	6
1			■	■		
2			■			
3	■	■		■		■
4	■		■		■	
5				■		■
6			■		■	

c)

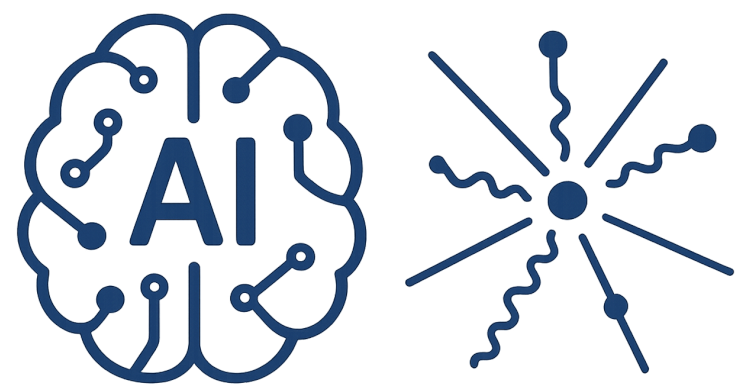
Node data, $\mathbf{X}$
 $D \times N$

	1	2	3	4	5	6
1	■	■	■	■	■	■
2	■	■	■	■	■	■
3	■	■	■	■	■	■
4	■	■	■	■	■	■
5	■	■	■	■	■	■

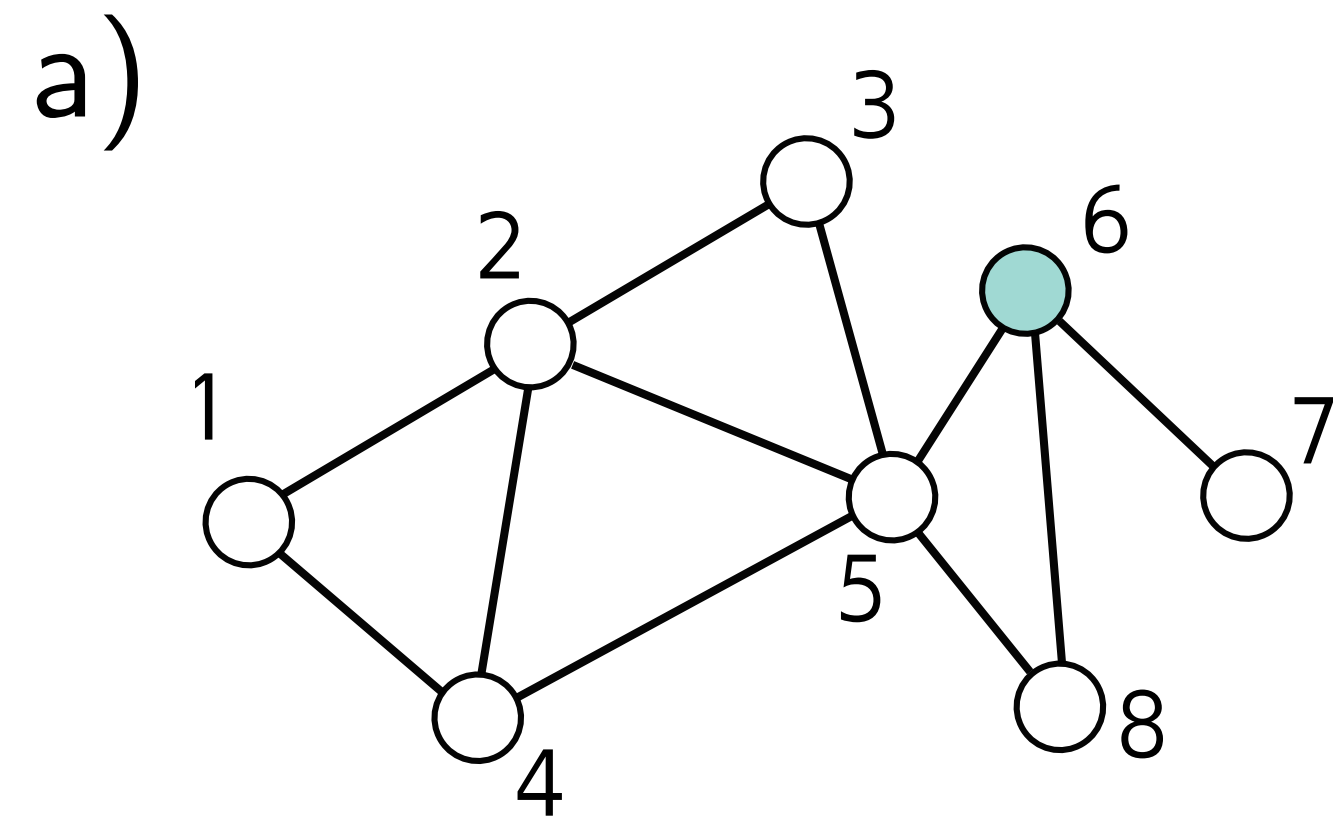
d)

Edge data, $\mathbf{E}$
 $D_E \times E$

	1	1	2	3	3	4	5
	3	4	3	4	6	5	6
1	■	■	■	■	■	■	■
2	■	■	■	■	■	■	■
3	■	■	■	■	■	■	■
4	■	■	■	■	■	■	■



Neighbors of Nodes



b)

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

c)

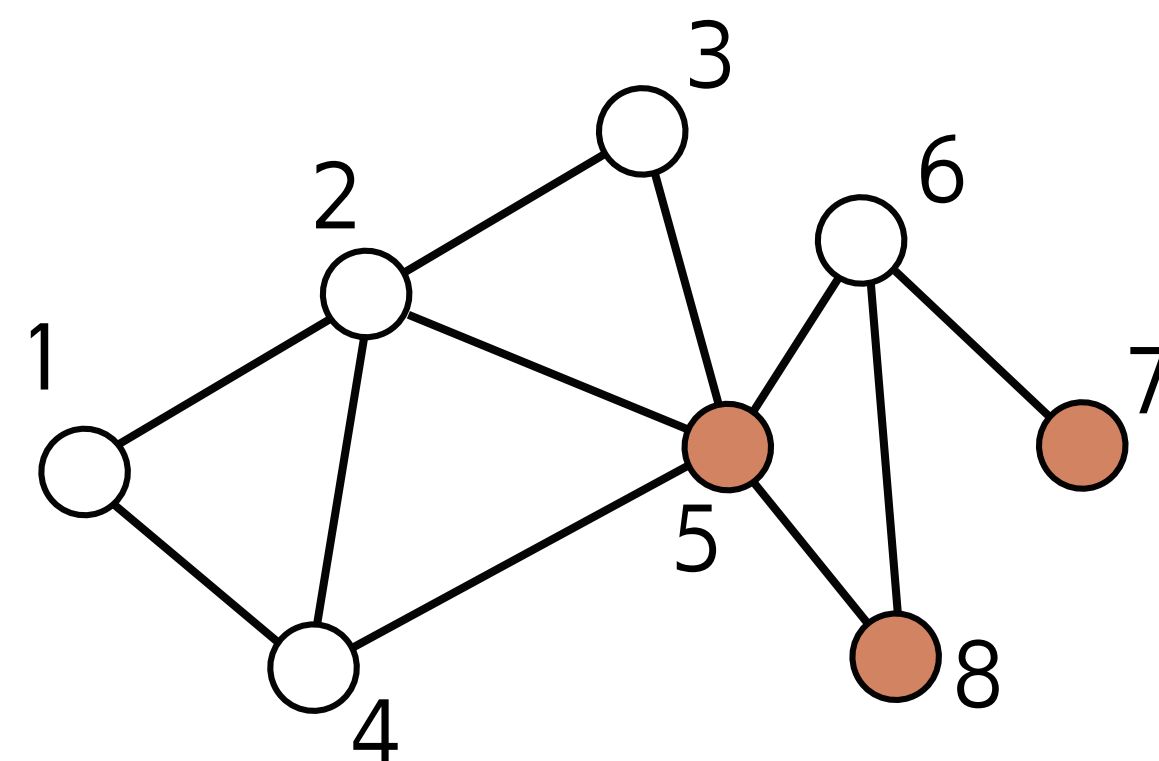
$$\mathbf{A}^2 = \begin{bmatrix} 2 & 1 & 1 & 1 & 2 & 0 & 0 & 0 \\ 1 & 4 & 1 & 2 & 2 & 1 & 0 & 1 \\ 1 & 1 & 2 & 2 & 1 & 1 & 0 & 1 \\ 1 & 2 & 2 & 3 & 1 & 1 & 0 & 1 \\ 2 & 2 & 1 & 1 & 5 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 3 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 1 & 1 & 2 \end{bmatrix}$$

d)

$$\mathbf{x} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

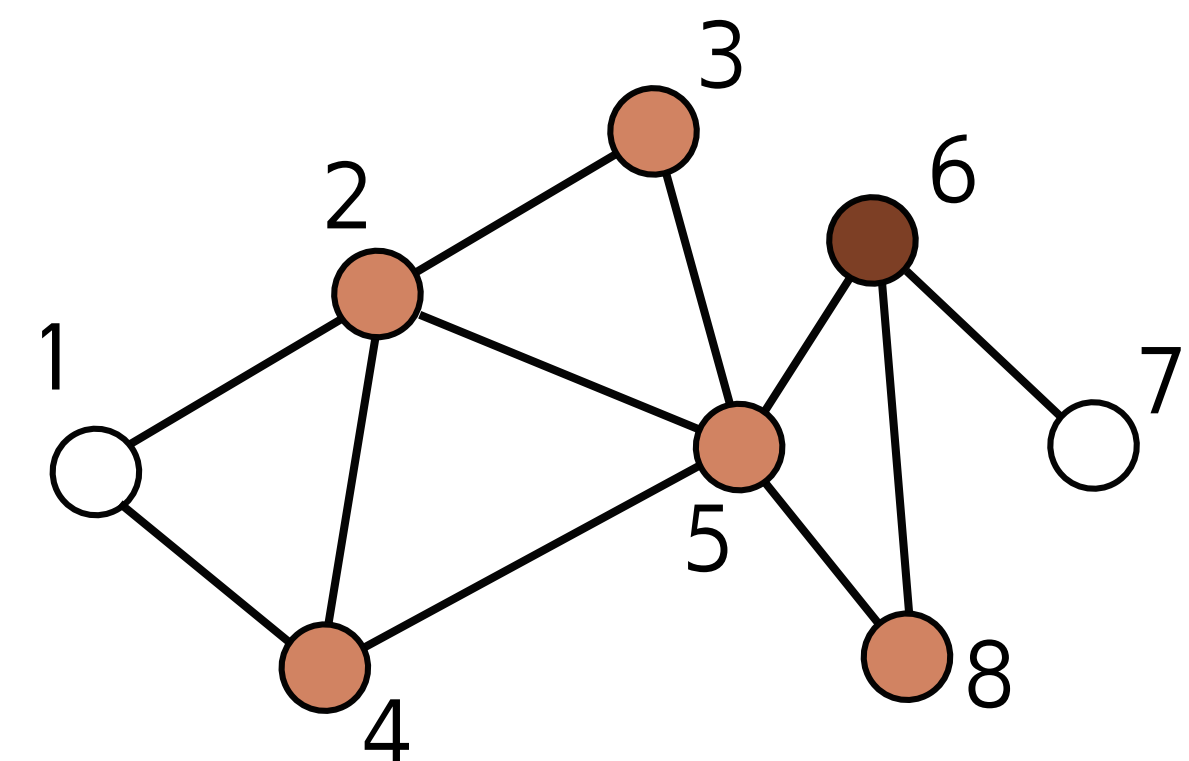
e)

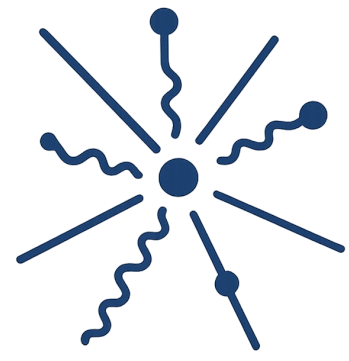
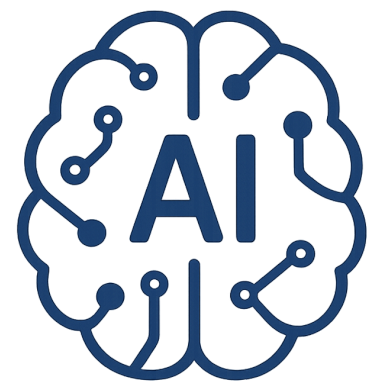
$$\mathbf{Ax} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$



f)

$$\mathbf{A}^2\mathbf{x} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 3 \\ 0 \\ 1 \end{bmatrix}$$





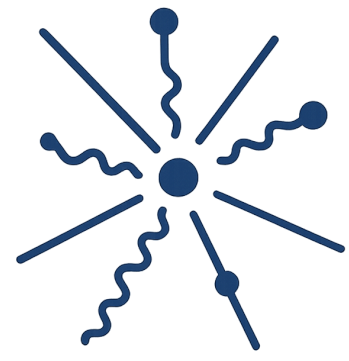
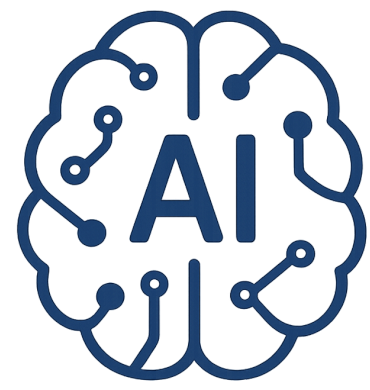
AI-Driven HEP 25

Graph Convolutional Networks

Assigns label (classification) or values (regression)

From the entire graph (Graph-level tasks)

Each node of the graph (Node-level tasks)



Graph Convolutional Networks

Assigns label (classification) or values (regression)

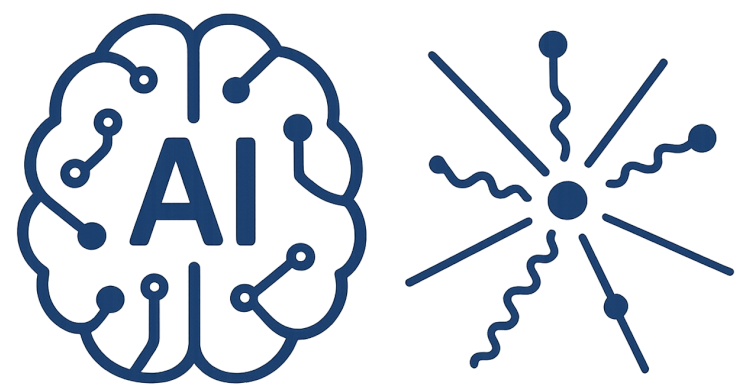
From the entire graph (Graph-level tasks)

Each node of the graph (Node-level tasks)

Edge prediction tasks

Network predicts whether or not there
should be an edge between nodes

Binary Classification



Graph Convolutional Networks

Assigns label (classification) or values (regression)

From the entire graph (Graph-level tasks)

Each node of the graph (Node-level tasks)

Edge prediction tasks

Network predicts whether or not there
should be an edge between nodes

Binary Classification

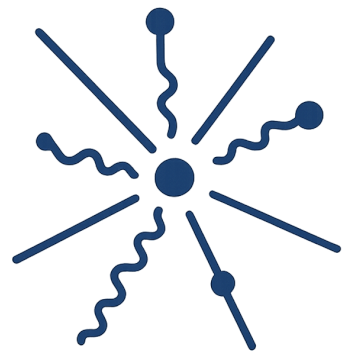
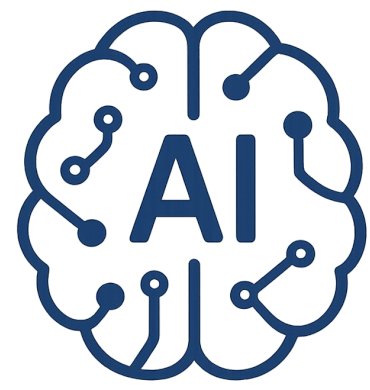
$$\mathbf{H}_1 = \mathbf{F}[\mathbf{X}, \mathbf{A}, \phi_0]$$

$$\mathbf{H}_2 = \mathbf{F}[\mathbf{H}_1, \mathbf{A}, \phi_1]$$

$$\mathbf{H}_3 = \mathbf{F}[\mathbf{H}_2, \mathbf{A}, \phi_2]$$

$$\vdots = \vdots$$

$$\mathbf{H}_K = \mathbf{F}[\mathbf{H}_{K-1}, \mathbf{A}, \phi_{K-1}],$$

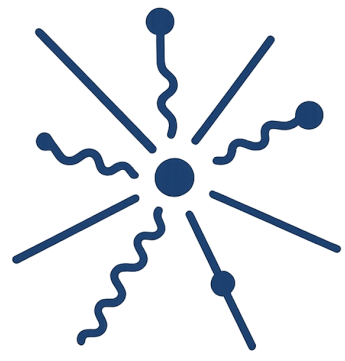
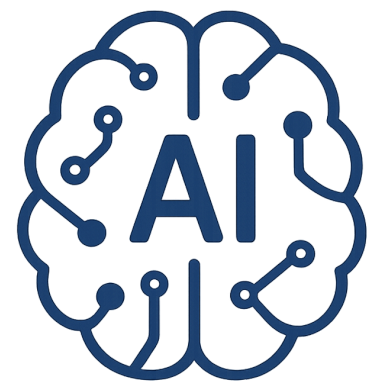


Graph Convolutional Networks

Convolutional layer for Images

$$z_i^{(l+1)} = f \left(\sum_j w_j z_j^{(l)} + b \right)$$

Sum is taken over a patch



Graph Convolutional Networks

Convolutional layer for Images

$$z_i^{(l+1)} = f \left(\sum_j w_j z_j^{(l)} + b \right)$$

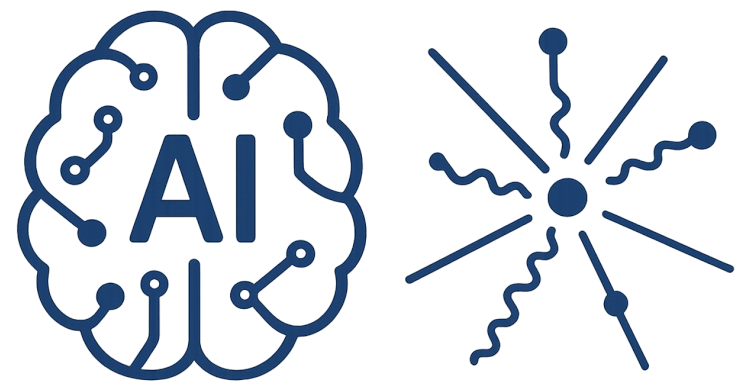
Sum is taken over a patch

It is not equivariant under reordering of
the nodes



Permutation

Node indexing in graphs is arbitrary

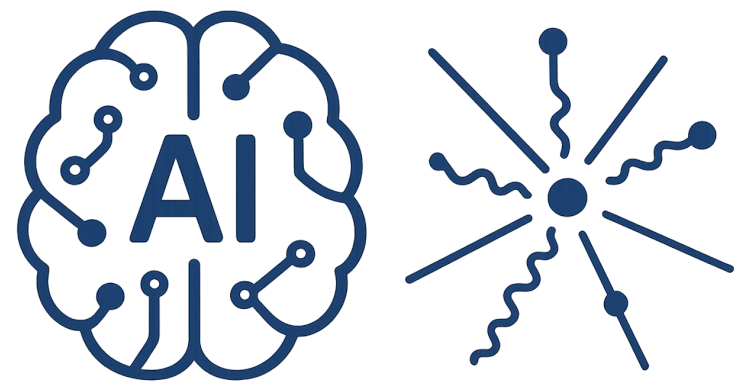


Permutation

Node indexing in graphs is arbitrary

$$(A, B, C, D, E) \rightarrow (C, \bar{E}, A, D, B)$$

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$



Permutation

Node indexing in graphs is arbitrary

$$(A, B, C, D, E) \rightarrow (C, E, A, D, B)$$

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Node n will be relabelled as node m

$$\tilde{\mathbf{X}} = \mathbf{P}\mathbf{X}$$

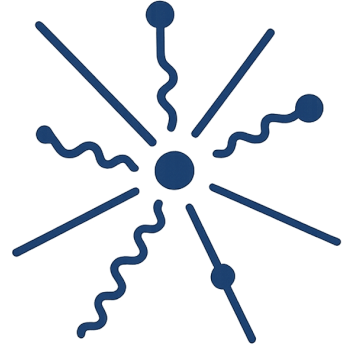
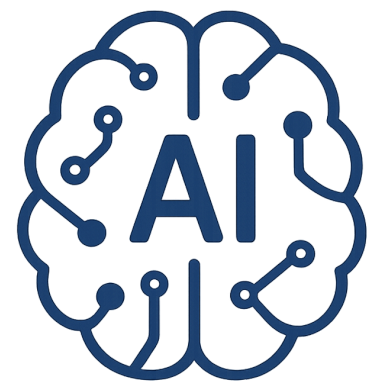
$$\tilde{\mathbf{A}} = \mathbf{P}\mathbf{A}\mathbf{P}^T$$



Equivariance

$$\mathbf{y}(\tilde{\mathbf{X}}, \tilde{\mathbf{A}}) = \mathbf{P}\mathbf{y}(\mathbf{X}, \mathbf{A})$$

node-level predictions

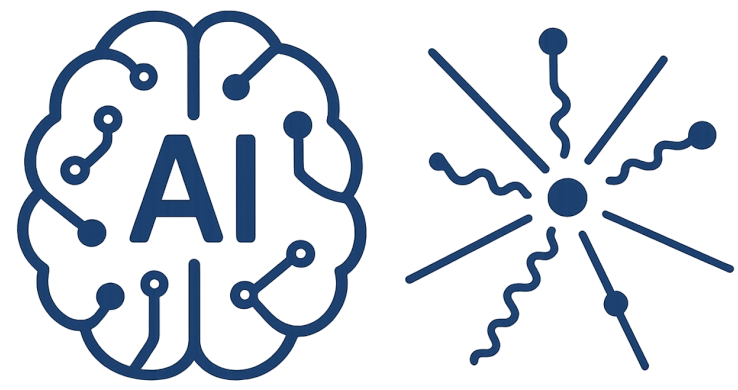


AI-Driven HEP 25

Invariance

$$y(\tilde{\mathbf{X}}, \tilde{\mathbf{A}}) = y(\mathbf{X}, \mathbf{A})$$

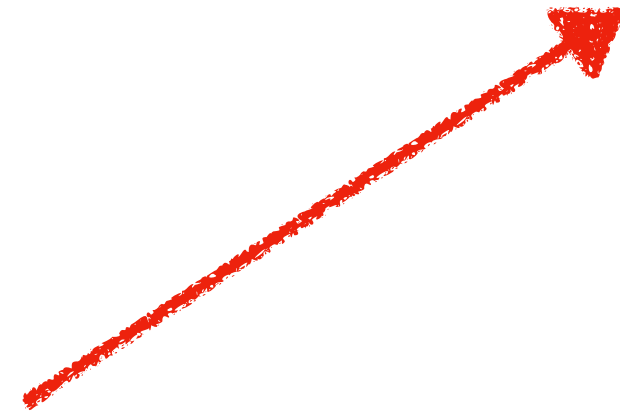
graph-level prediction



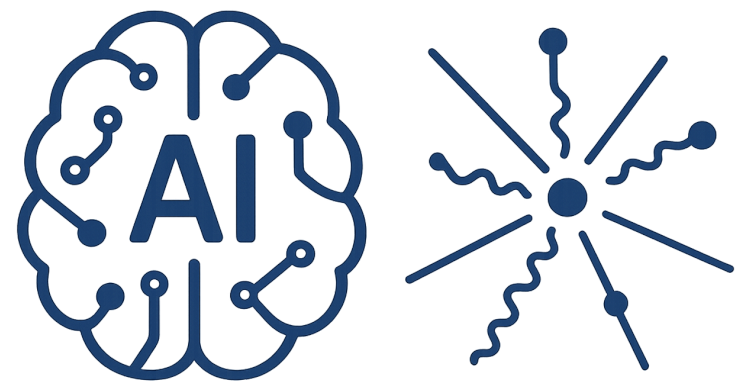
Messages Passing

$$z_i^{(l+1)} = f \left(w_{\text{neigh}} \sum_{j \in \mathcal{N}(i)} z_j^{(l)} + w_{\text{self}} z_i^{(l)} + b \right)$$

Neighbouring Nodes



Shared Parameters



Messages Passing

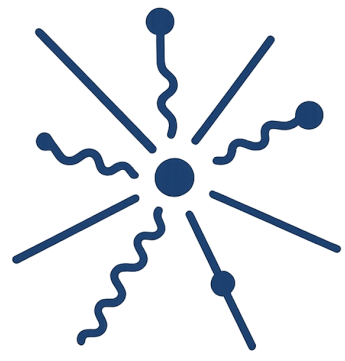
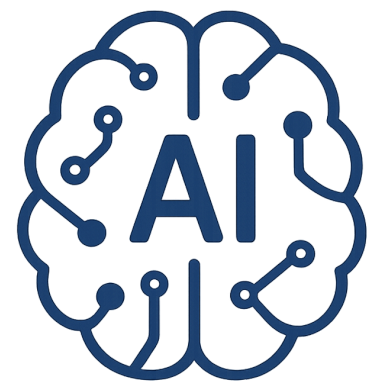
$$z_i^{(l+1)} = f \left(w_{\text{neigh}} \sum_{j \in \mathcal{N}(i)} z_j^{(l)} + w_{\text{self}} z_i^{(l)} + b \right)$$

Shared Parameters

Neighbouring Nodes

Invariant to any permutation of the nodes labels

Equivariant under node reordering



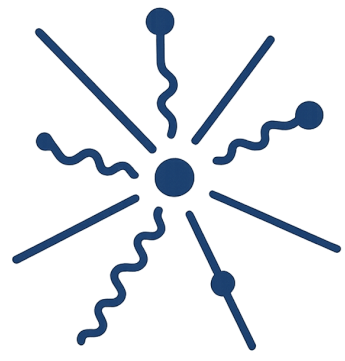
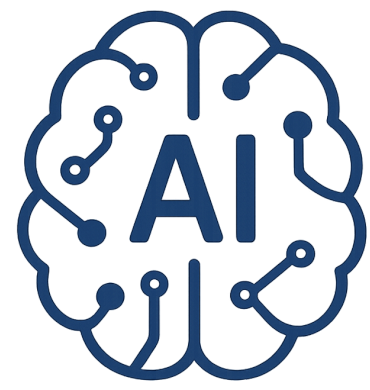
Message-Passing Neural Network

$$\mathbf{z}_n^{(l)} = \text{Aggregate} \left(\left\{ \mathbf{h}_m^{(l)} : m \in \mathcal{N}(n) \right\} \right)$$

It is a differentiable function

Number of neighbouring nodes are not fixed

Invariant under the permutations of neighbouring nodes



Message-Passing Neural Network

$$\mathbf{z}_n^{(l)} = \text{Aggregate} \left(\left\{ \mathbf{h}_m^{(l)} : m \in \mathcal{N}(n) \right\} \right)$$

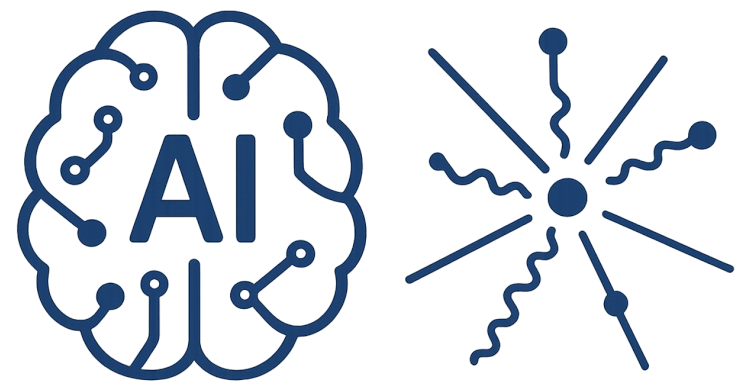
It is a differentiable function

Number of neighbouring nodes are not fixed

Invariant under the permutations of neighbouring nodes

$$\mathbf{h}_n^{(l+1)} = \text{Update} \left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)} \right)$$

operates in parallel for every node



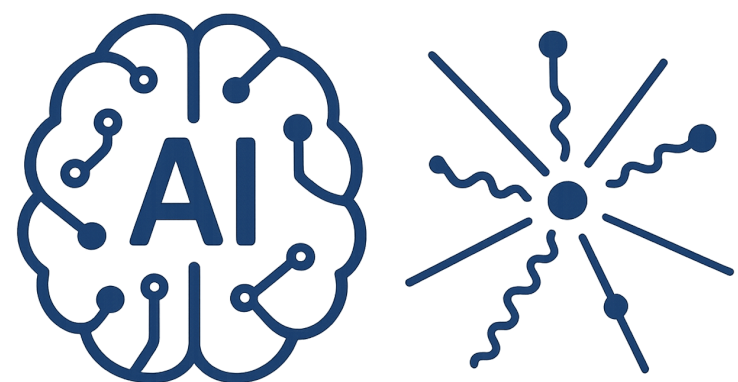
Aggregation Operators

$$\text{Aggregate} \left(\{ \mathbf{h}_m^{(l)} : m \in \mathcal{N}(n) \} \right) = \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)}.$$

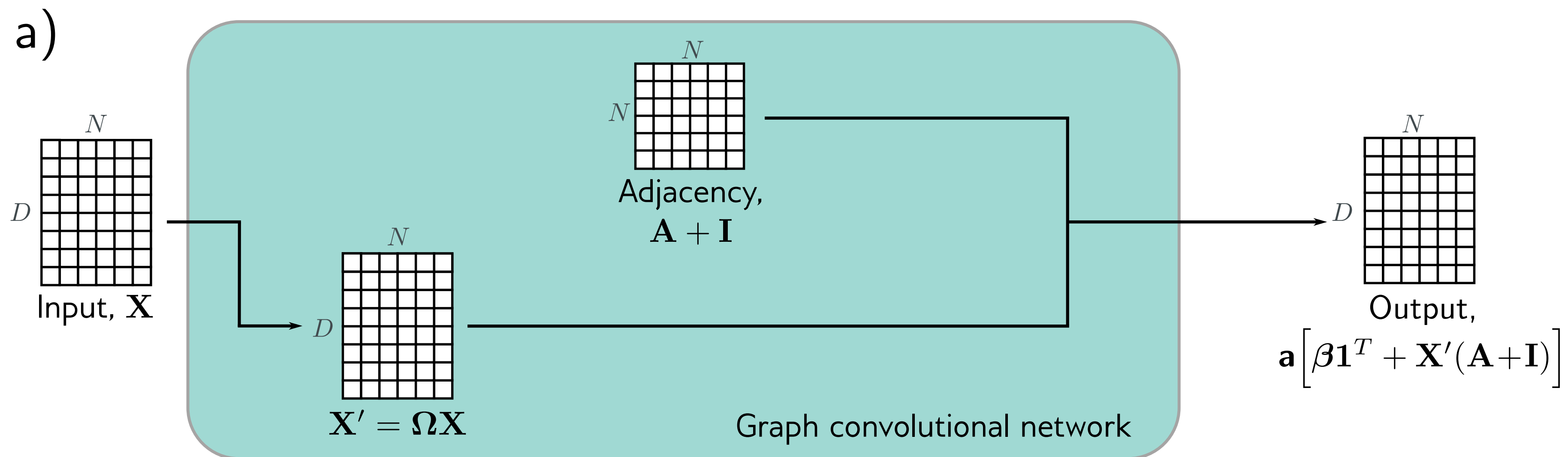
$$\text{Aggregate} \left(\{ \mathbf{h}_m^{(l)} : m \in \mathcal{N}(n) \} \right) = \frac{1}{|\mathcal{N}(n)|} \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)}$$

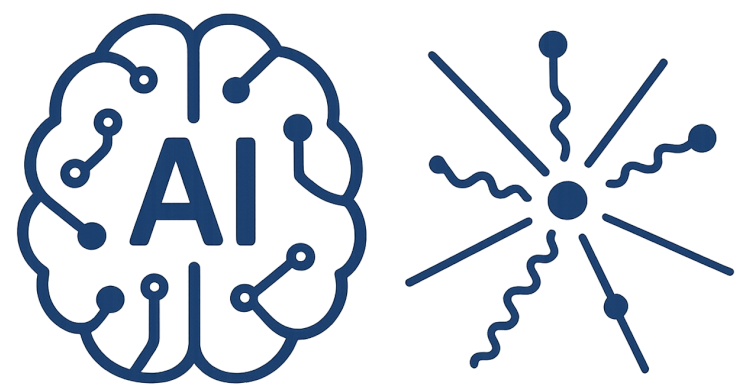
$$\text{Aggregate} \left(\{ \mathbf{h}_m^{(l)} : m \in \mathcal{N}(n) \} \right) = \text{MLP}_{\theta} \left(\sum_{m \in \mathcal{N}(n)} \text{MLP}_{\phi}(\mathbf{h}_m^{(l)}) \right)$$

$$\text{Aggregate} \left(\{ \mathbf{h}_m^{(l)} : m \in \mathcal{N}(n) \} \right) = \sum_{m \in \mathcal{N}(n)} A_{nm} \mathbf{h}_m^{(l)}$$

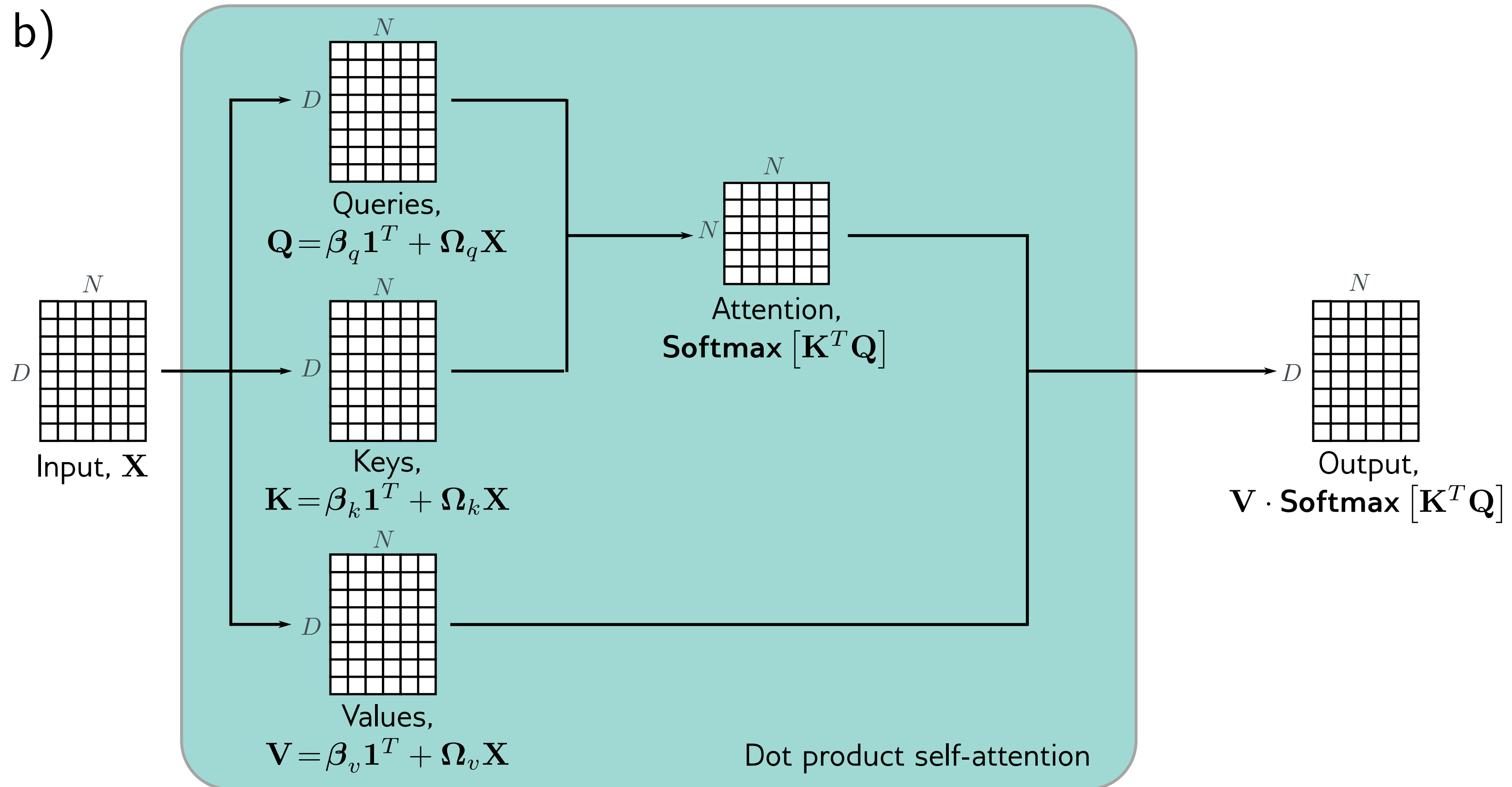


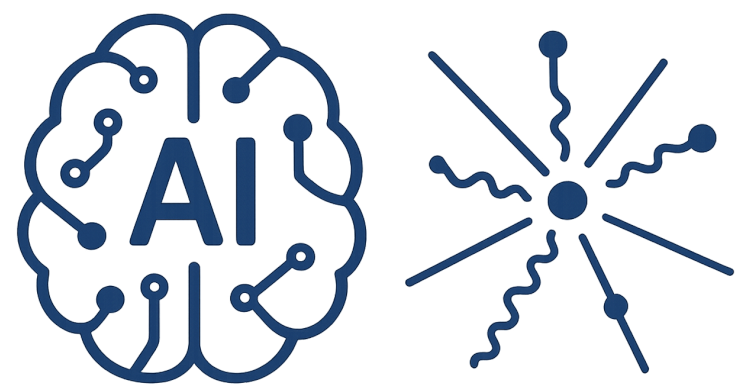
Graph Attention Networks



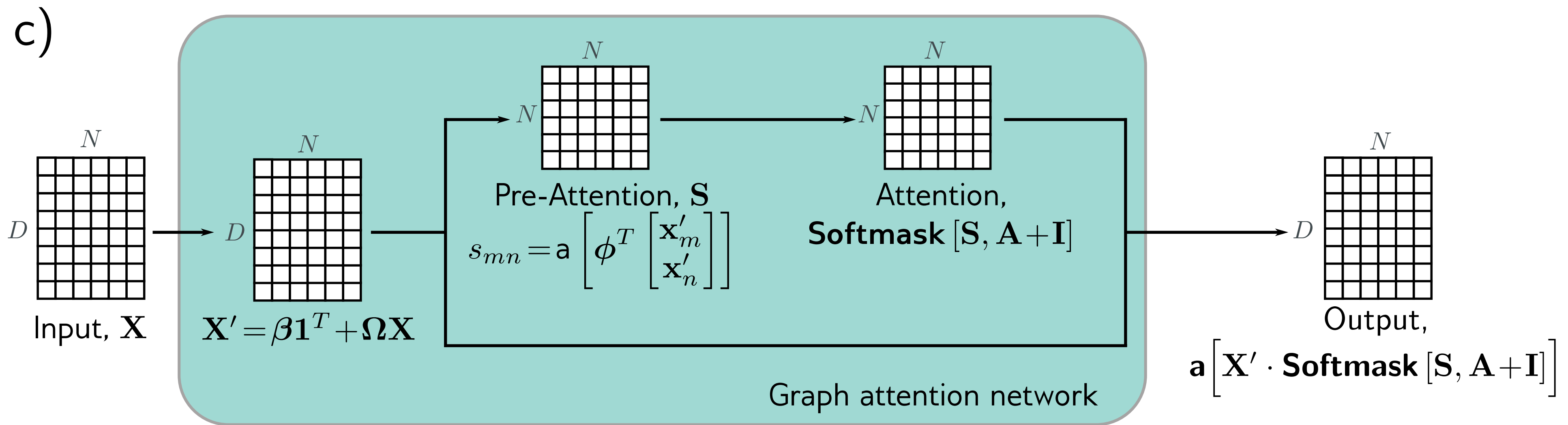


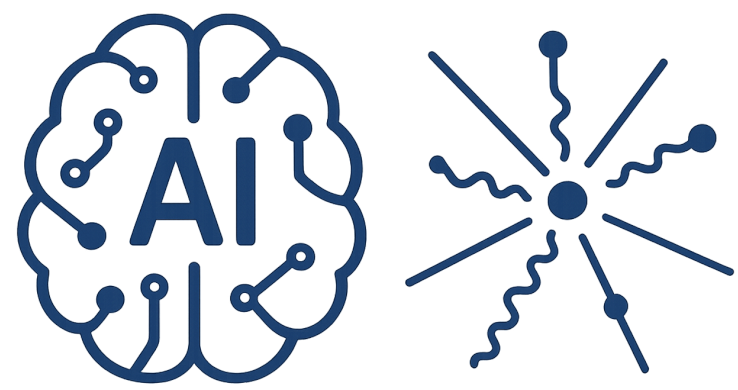
Graph Attention Networks





Graph Attention Networks





Graph Networks

